Week 7 Technical Report - Randomized Algorithms

Project repository (complete source, tests, benchmarks and results):
https://github.com/RDeibel2025/CTX-Advanced-Algorithms
Randomized algorithms: src/randomized/ · demonstration: examples/week7_demo.py
Benchmark: benchmarks/week7_randomized_benchmark.py

Robert Deibel · CSC 5300 Advanced Algorithms · Concordia University Texas · Fall 2026

📎 Complete project repository

https://github.com/RDeibel2025/CTX-Advanced-Algorithms

1. Executive Summary

I implemented randomized QuickSort and QuickSelect, the Fermat and Miller-Rabin primality tests, a Bloom filter and MinHash, each against a deterministic or exact baseline. The report measures, for each algorithm, how much extra work buys how much extra confidence. A Miller-Rabin round cost 2.0 to 13.3 microseconds and cut the error bound to a quarter; a Fermat round cost the same but on the Carmichael number 561 multiplied the error by only 0.57. Each extra bit per item multiplied a Bloom filter's false-positive rate by 0.624, and quadrupling a MinHash signature halved its error. Randomized QuickSort and QuickSelect were always correct; their random draws buy a narrow comparison-count spread on every input.

2. Methodology

Machine and timing. Apple M2 Max, 64 GB RAM, macOS 14.5 (arm64), CPython 3.12.4. Runs are timed with `time.perf_counter` with garbage collection paused. QuickSort and QuickSelect have 30 timed runs per point after warm-ups; median of medians 5; the sorted-input fixed pivot 5 (3 from n = 5,000); the k = 20 primality tests 30; trial division 5 (3 above 10^12).

Seeds. The master seed is 42 (`--seed`). Every input, pivot sequence, base choice and hash family gets its own sub-seed from `derive_seed`, the first 8 bytes of a blake2b digest of the master seed and labels; the CSV records 179 distinct seeds. Each algorithm uses its own `random.Random` instance. The Bloom filter and MinHash hash with seeded blake2b, not Python's `hash()`, which `PYTHONHASHSEED` salts per process. Tests confirm identical filter bits under two `PYTHONHASHSEED` values and identical non-timing CSV rows across two runs.

Inputs. QuickSort: distinct random integers, n = 100 to 100,000, and sorted input to 10,000; comparison counts over 1,000 seeds at n = 1,000 and 200 at 10,000. Selection: minimum, median and maximum, n = 1,000 to 100,000. Primality: seven Carmichael numbers from 561 to 8,911, plus 2,047 and 1,000,001, with 10,000 trials at k = 1, 5, 10 and 20; the prediction is the exact liar fraction to the k-th power. Speed: the first prime above 10^d, d = 6 to 14, and 2^61 - 1. Bloom: m = 10,000 and k = 7 with 100 to 5,000 members, and 2 to 20

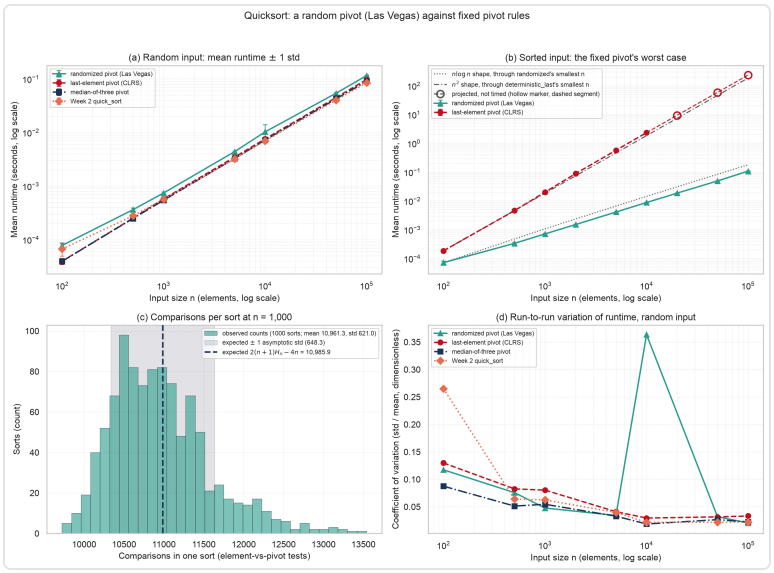
bits per item at the optimal k; 100,000 non-member queries each. MinHash: five simulated pairs with exact Jaccard 0.10 to 0.90 and four word 3-gram pairs from an original paragraph and seeded edits of it; k = 16 to 512, 50 seeds each. Rates carry 95% Wilson intervals, and zero counts also the rule of three.

Baseline. Week 2's `quick_sort` was randomized by requirement, so I built last-element and median-of-three versions this week and timed Week 2's code as a reference.

Projections and rechecks. Four CSV rows are marked `projected`: the last-element pivot on sorted input at n = 20,000, 50,000 and 100,000 (46.8 ns per comparison measured at 10,000, times n(n-1)/2), and trial division at 2^61 - 1 (the per-division time at 10^14, times 759,250,124 divisions). On 3,215,031,751 the 10,000-trial rate at k = 5 exceeded the 4^-5 bound, so I reran k = 1 and 5 with 200,000 trials and kept both.

3. Results

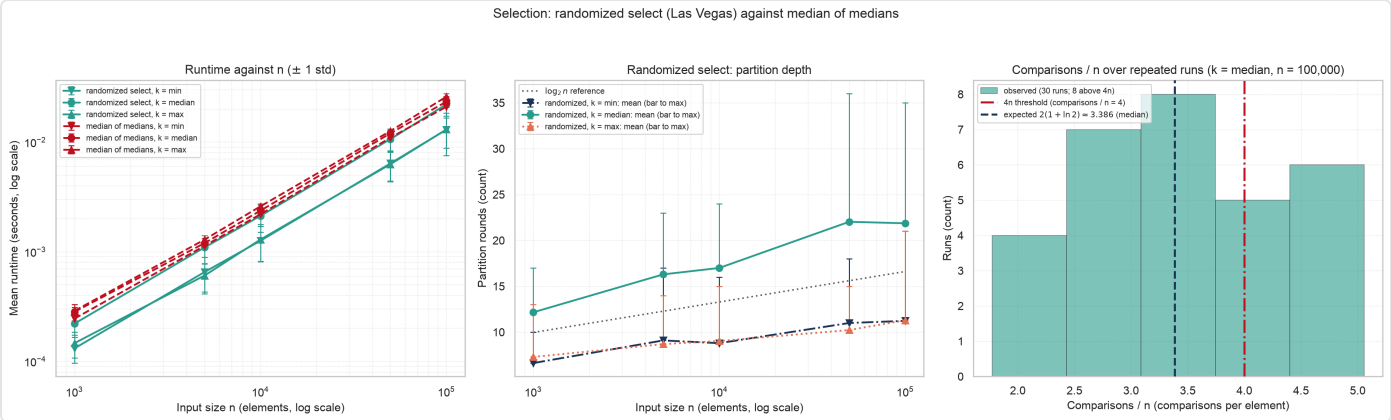
3.1 QuickSort



On random input at n = 100,000, randomized QuickSort took 115.4 ± 2.4 ms, against 97.3 ± 3.2 ms for the last-element pivot (1.186x), 90.3 ms for median of three and 84.7 ms for Week 2. Its coefficient of variation was 0.021 to 0.117, except 0.364 at n = 10,000, from one 29.9 ms run against an 8.9 ms minimum. At n = 1,000 the mean comparison count was 10,961.3 against the expected 2(n+1)H_n - 4n = 10,985.9 (-0.22%), with standard deviation 621.0 against 0.6483n = 648.3. At n = 10,000 the figures were 155,516.9 against 155,771.7 and 6,764.1 against 6,482.8, so the spread fell from 5.7% to 4.3% of the mean.

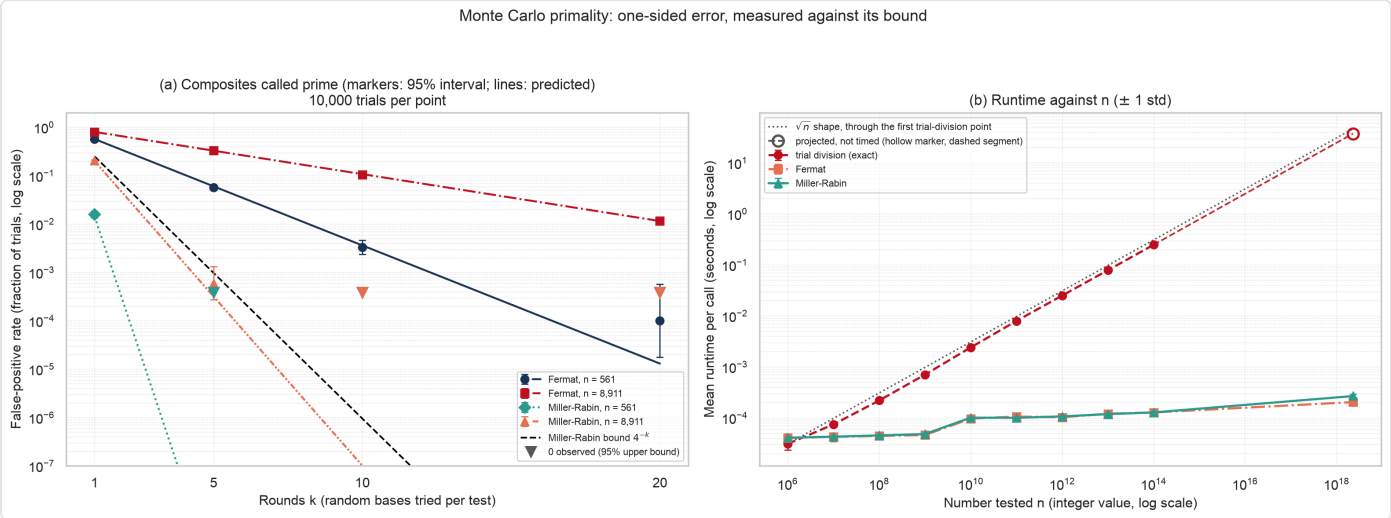
On sorted input at n = 10,000 the last-element pivot made exactly 49,995,000 comparisons in 2,337.9 ms; randomized QuickSort took 8.7 ms, 268x less. At 20,000, 50,000 and 100,000 the fixed pivot projects to 9.4 s, 58.5 s and 233.8 s, while the randomized version measured 18.5, 49.2 and 107.6 ms.

3.2 Selection



QuickSelect was correct in 30 of 30 runs at every point. It averaged 3.32 to 3.74 comparisons per element for the median, against the expected 2 + 2 ln 2 = 3.386, and 1.80 to 2.21 for the minimum and maximum. Between 0.53 and 0.83 of median runs finished within 4n comparisons. Its timing coefficient of variation was 0.21 to 0.42, against 0.01 to 0.17 for median of medians, which took 1.06 to 1.28x as long for the median and 1.62 to 2.13x for the minimum and maximum.

3.3 Primality

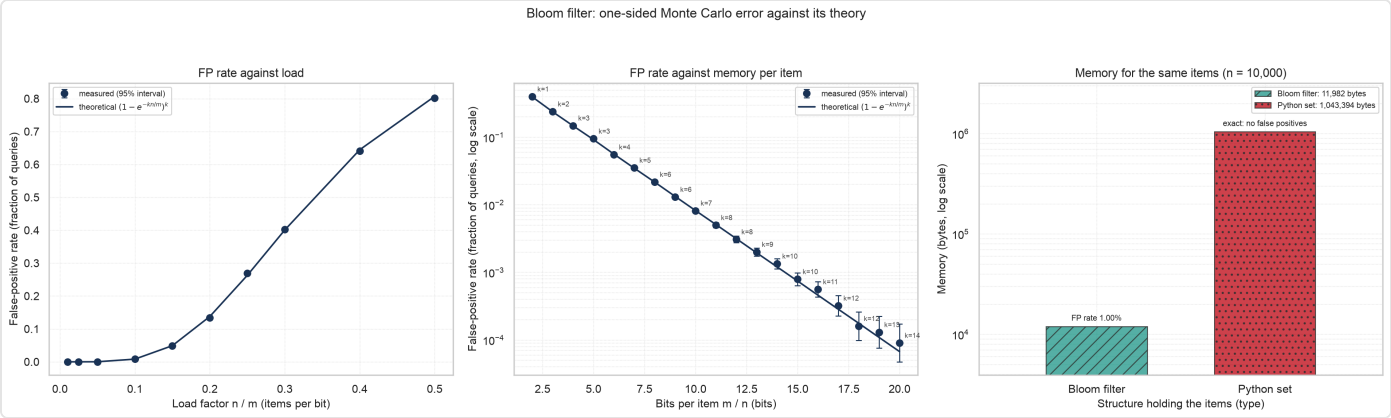


On 561, Fermat was fooled at 0.5709 [0.5612, 0.5806] at k = 1, 0.0564 at k = 5, 0.0033 at k = 10 and 0.0001 at k = 20, against predictions of 0.570, 0.0601, 0.00361 and 1.31e-5. At k of 5 or more Miller-Rabin made 6 false positives in 270,000 trials, all on 8,911 at k = 5: 0.0006 [0.00028, 0.00131] against a predicted 3.19e-4. Each of the 26 zero-count points has a 95% upper bound of 3.84e-4.

Twenty Miller-Rabin rounds took 40.6 microseconds at 1,000,003 (2.03 per round), 127.5 at 10^14 and 266.8 at 2^61 - 1 (13.34 per round). Trial division was faster at 10^6 (0.76x) but 1,938x slower at 10^14, and projects to 37.5 s at 2^61 - 1.

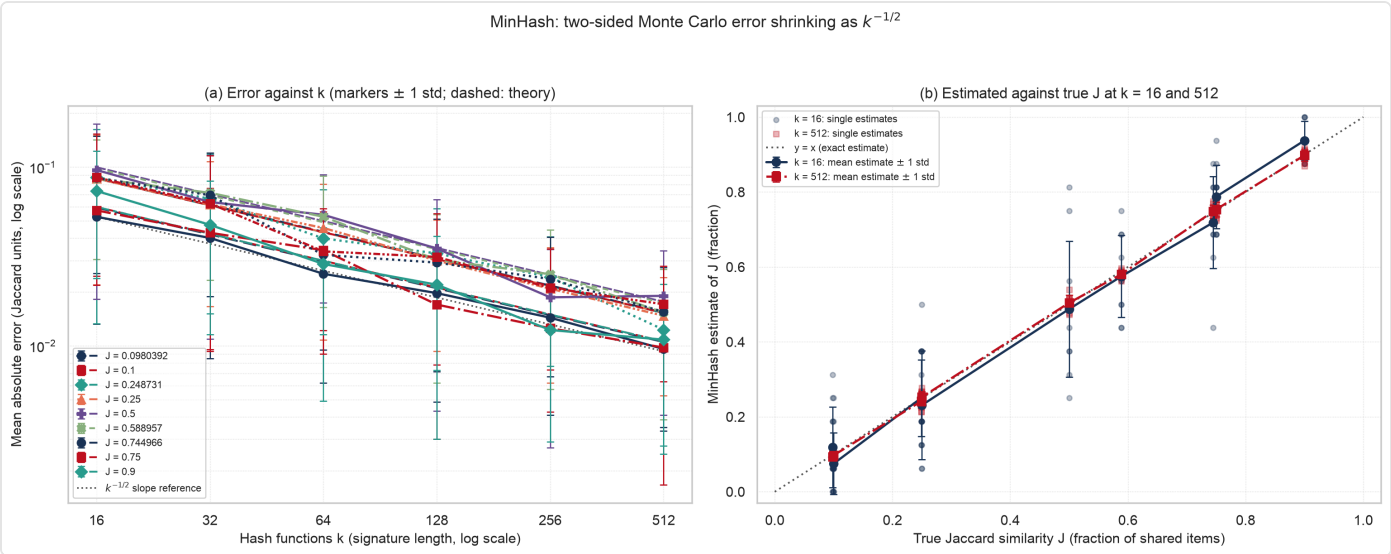
With fixed bases {2, 3, 5, 7}, Miller-Rabin always calls 3,215,031,751 prime. With random bases, 200,000 trials gave 0.24762 [0.24573, 0.24952] at k = 1 and 0.000915 [0.000792, 0.001057] at k = 5, against an exact strong-liar fraction of 0.2480 and its fifth power 9.38e-4. The 10,000-trial k = 5 value, 0.0016, was sampling noise.

3.4 Bloom filter



There were no false negatives. At load 0.1 the false-positive rate was 0.00882 [0.00826, 0.00942] against a theoretical 0.00819. Theory fell inside the interval at only 4 of 10 load points because each point is one filter, whose rate is its own fill ratio to the power k, while the formula averages over filters. Against fill^k the mean relative gap was 0.8%, against 2.3% for the formula. In the bits sweep theory fell inside 17 of 19 intervals, and each extra bit per item multiplied the rate by 0.6240 against 0.6181 in theory. A filter sized for 1% at 10,000 items used 11,982 bytes, 87x less than a Python set of the same strings (1,043,394 bytes).

3.5 MinHash



Mean absolute error over the nine pairs fell from 0.0794 at k = 16 to 0.0385 at 64, 0.0192 at 256 and 0.0138 at 512, against 0.0803, 0.0402, 0.0201 and 0.0142 from sqrt(J(1-J)/k). The log-log slope was -0.512 against -0.5, and the largest bias at k = 512 was 0.0034.

4. Discussion

The Las Vegas algorithms put their randomness in the running time and the Monte Carlo algorithms put it in the answer. QuickSort and QuickSelect were always correct, and randomization cost them spread. Miller-Rabin's "composite" and a Bloom filter's "absent" are always right, so their error is one-sided, and each extra round or bit multiplies it by a constant: error falls exponentially in the work. MinHash errs in both directions, so extra work only averages it down, at k^(-1/2). Twenty rounds took Miller-Rabin to a 4^-20 bound, while 32 times more MinHash hashing cut error 5.76x.

Randomization defeats adversarial inputs because the bad case depends on choices the adversary cannot see. A fixed last-element pivot turns sorted input into n(n-1)/2 comparisons; with a random pivot, sorted input cost 161,897 comparisons at n = 10,000, near the 155,772 expected for random data. 3,215,031,751 defeats fixed bases, but random bases catch it with probability 0.752 per round. That protection needs a secret seed: the published seed 42 makes these experiments reproducible, and would let an adversary predict every pivot.

Randomized QuickSort was 18.6% slower than the last-element pivot on random input in exchange for removing the sorted-input case. The Bloom filter used 87x less memory than a set for 1% false positives and no deletion. In practice, cryptographic key generation tests random candidates with Miller-Rabin, databases and caches check Bloom filters before costly lookups, and near-duplicate detection in web crawls and machine-learning training data uses MinHash.

5. Conclusion

Moving from exactness to expected efficiency trades a worst case that some input can force for an expected bound that holds on every input. For each algorithm here, confidence rose predictably with the work spent. At 2^61 - 1, twenty Miller-Rabin rounds took 266.8 microseconds for an error bound of 4^-20, while certainty by trial division is projected at 37.5 seconds.

References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press. §7.3-7.4 A randomized version of quicksort and its analysis; §9.2-9.3 Selection in expected and worst-case linear time.
- Amakobe, M. (2025). *Advanced Computational Algorithms*, Ch. 7 (open textbook).
- Rabin, M. O. (1980). Probabilistic algorithm for testing primality. *Journal of Number Theory, 12*(1), 128-138.
- Bloom, B. H. (1970). Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM, 13*(7), 422-426.
- Broder, A. Z. (1997). On the resemblance and containment of documents. *Proceedings of Compression and Complexity of Sequences 1997*, 21-29.
- Kirsch, A., & Mitzenmacher, M. (2006). Less hashing, same performance: Building a better Bloom filter. *ESA 2006*, LNCS 4168, 456-467.

AI use

Claude (Claude Code) drafted the code, tests and this report's first draft from a specification I wrote from the instructions and required reading, including the hand-checked values the tests assert. Every measurement comes from running the benchmark on my machine, and every number in this report is printed from the CSV by `tools/week7_facts.py`. Full disclosure: `docs/AI_USE.md`.